

1. Import Required Libraries

```
from keras.preprocessing.image import ImageDataGenerator
from keras.applications import MobileNetV2
from keras.layers import Input, Dense, Flatten, Dropout, AveragePooling2D
from keras.models import Model
from keras.optimizers import Adam
from keras.applications.mobilenet_v2 import preprocess_input
from keras.preprocessing.image import img_to_array, load_img
from keras.utils import to_categorical
from sklearn.preprocessing import LabelBinarizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report
from imutils import paths
import matplotlib.pyplot as plt
import numpy as np
import argparse
import os
```

Explanation:

Import all necessary Python libraries for deep learning, data processing, plotting, and command-line argument parsing.

2. Parse Command-Line Arguments

```
# construct the argument parser and parse the arguments
ap = argparse.ArgumentParser()
ap.add_argument("-d", "--dataset", required=True,
    help="path to input dataset")
ap.add_argument("-p", "--plot", type=str, default="plot.png",
    help="path to output loss/accuracy plot")
ap.add_argument("-m", "--model", type=str,
    default="mask_detectorch.model",
    help="path to output face mask detector model")
args = vars(ap.parse_args())
```

Explanation:

Set up the command-line interface to specify the dataset location, plot output file, and trained model save location.

3. Set Key Parameters

```
# initialize the initial learning rate, number of epochs to train for,
# and batch size
INIT_LR = 1e-4
EPOCHS = 20
BS = 32
```

Explanation:

Define learning rate, how many times the model sees the data, and how many images to process at once during training.

4. Load Images and Labels

```
imagePaths = list(paths.list_images(args["dataset"]))
data = []
labels = []

# Loop over the image paths
for imagePath in imagePaths:
    # extract the class label from the filename
    label = imagePath.split(os.path.sep)[-2]

    # load the input image (224x224) and preprocess it
    image = load_img(imagePath, target_size=(224, 224))
    image = img_to_array(image)
    image = preprocess_input(image)

    # update the data and labels lists, respectively
    data.append(image)
    labels.append(label)
```

Explanation:

Go through all image files in the dataset folder, load each image, resize it, preprocess it, and store both the image data and its label in lists.

5. Prepare Data for Training

```
# convert the data and labels to NumPy arrays
data = np.array(data, dtype="float32")
labels = np.array(labels)

# perform one-hot encoding on the labels
lb = LabelBinarizer()
labels = lb.fit_transform(labels)
labels = to_categorical(labels)
```

Explanation:

Convert images and labels to NumPy arrays. Encode the label text into numbers and then into categories for training.

6. Split Data into Training and Testing Sets

```
(trainX, testX, trainY, testY) = train_test_split(data, labels,
                                                    test_size=0.20, stratify=labels, random_state=42)
```

Explanation:

Divide the images and labels into training set (80%) and testing set (20%), making sure each label is evenly distributed.

7. Data Augmentation Setup

```
aug = ImageDataGenerator(  
    rotation_range=20,  
    zoom_range=0.15,  
    width_shift_range=0.2,  
    height_shift_range=0.2,  
    shear_range=0.15,  
    horizontal_flip=True,  
    fill_mode="nearest")
```

Explanation:

Build a data augmentation tool to create slightly modified versions of the original images to help the model generalize better.

8. Load Pretrained Base Model (MobileNetV2)

```
# Load the MobileNetV2 network, ensuring the head FC layer sets are  
# left off  
baseModel = MobileNetV2(weights="imagenet", include_top=False,  
    input_tensor=Input(shape=(224, 224, 3)))
```

Explanation:

Import the MobileNetV2 model (already trained on millions of images) and set it up to process images.

9. Build Custom Head for Mask Detection

```
headModel = baseModel.output  
headModel = AveragePooling2D(pool_size=(7, 7))(headModel)  
headModel = Flatten(name="flatten")(headModel)  
headModel = Dense(128, activation="relu")(headModel)  
headModel = Dropout(0.5)(headModel)  
headModel = Dense(2, activation="softmax")(headModel)
```

Explanation:

Add new layers on top of the pretrained base to allow the model to learn to distinguish between mask and no-mask face images.

10. Combine Base and Head into Final Model

```
model = Model(inputs=baseModel.input, outputs=headModel)
```

Explanation:

Connect the base and new head making a complete model ready to train for the mask detector task.

11. Freeze Base Model Layers

```
for layer in baseModel.layers:  
    layer.trainable = False
```

Explanation:

Lock the base model weights so only the custom head layers are updated during training.

12. Compile the Model

```
print("[INFO] compiling model...")  
opt = Adam(lr=INIT_LR, decay=INIT_LR / EPOCHS)  
model.compile(loss="binary_crossentropy", optimizer=opt,  
              metrics=["accuracy"])
```

Explanation:

Set up the optimizer and compile the model with loss and accuracy metrics for training.

13. Train the Model

```
H = model.fit(  
    aug.flow(trainX, trainY, batch_size=BS),  
    steps_per_epoch=len(trainX) // BS,  
    validation_data=(testX, testY),  
    validation_steps=len(testX) // BS,  
    epochs=EPOCHS)
```

Explanation:

Train the new head layers using augmented images while monitoring validation accuracy after each epoch.

14. Make Predictions with the Model

```
predIdxs = model.predict(testX, batch_size=BS)  
predIdxs = np.argmax(predIdxs, axis=1)
```

Explanation:

Use the trained model to predict mask/no-mask for test images, selecting the most probable class for each image.

15. Evaluate Model Performance

```
print(classification_report(testY.argmax(axis=1), predIdxs, target_names=lb.classes_))
```

Explanation:

Show a detailed report with precision, recall, and f1-score for both classes (mask, no mask) on the test data.

16. Save the Model to Disk

```
print("[INFO] saving mask detector model...")
model.save(MODEL_PATH, save_format="h5")
```

Explanation:

Store the trained model as a file so it can be loaded and used later.

17. Plot Training History

```
N = EPOCHS
plt.style.use("ggplot")
plt.figure()
plt.plot(np.arange(0, N), H.history["loss"], label="train_loss")
plt.plot(np.arange(0, N), H.history["val_loss"], label="val_loss")
plt.plot(np.arange(0, N), H.history["accuracy"], label="train_accuracy")
plt.plot(np.arange(0, N), H.history["val_accuracy"], label="val_accuracy")
plt.title("Training Loss and Accuracy")
plt.xlabel("Epoch #")
plt.ylabel("Loss/Accuracy")
plt.legend(loc="lower left")
plt.savefig(PLOT_PATH)
plt.close()
```

Explanation:

Create and save a graph showing how the model's accuracy and loss changed during training.